A Project On

A Secure Erasure Code-Based Cloud Storage System with Secure Data Forwarding

ABSTRACT

Cloud storage is a service model in which data is maintained, managed and backed up remotely and made available to users over a network. Having your data stored offsite in the cloud makes it accessible from anywhere without the hassle of maintaining your own local storage and file-serving systems.Storing the data into a third party's cloud system causes concern over data confidentiality. In this cloud some general encryption schemes protect data confidentiality, but also limit the functionality of the storage system because a few operations are supported over encrypted data. The main objective of this project is constructing a secure storage system that supports multiple functions.In this project I propose a re-encryption scheme and integrate it with an erasure code such that a secure storage system is formulated. The storage system supports secure and robust data storage and retrieval.The main technical contribution is that the re-encryption scheme supports encoding operations over encrypted messages as well as forwarding operations over encoded and encrypted messages. This method fully integrates encrypting, encoding, and forwarding. Here erasure coding splits a message into multiple segments and then re-encryption scheme is used to re-encrypt those segmented messages and after that those re-encrypted messages are stored in the storage servers.Erasure encoding supports the forwarding scheme and is applicable in this storage system. An erasure coding is used to ensure the data robustness in the cloud storage system.

Keywords: Storage servers; Re-encryption; Erasure coding; Encryption; Decryption; Secure Data Forwarding.

Table of Contents

ABSTRACT……2

Introdiction……5
1.1 Background……6
1.2 Motivation……...6
1.3 Problem Statement……..7
1.4 Aims and Objectives……..……...……7
1.5 Limitations of this Project……..8

Literature review……9
2.1 Cloud Storage System……...10
2.2 Erasure Coding Techniques……..10
2.3 Secure Data Forwarding Mechanisms……..10

Existing System……11
3.1 General Overview of Existing System……..12
3.2 Disadvantages of the Existing System……..12

Proposed System Methodology……...……...13
4.1 Overview of the Proposed System……14
4.2 System Architecture……..15
4.3 Data Flow Diagram……...16
4.4 Advantages of the Proposed System……17
4.5 Components and Modules……...……17
4.5.1 Cloud Data Storage Module……17
4.5.2 Data Encryption Module……17
4.5.3 Erasure-coding module……...18
4.5.4 Re-encryption Module……18
4.5.6 Data forwarding and retrieving module……..18

Requirements……19
5.1 Requirements……20
5.1.1 Hardware requirements……..20
5.1.2 Software Requirements……..20

Implementation……21
7.1 Bash script……22
7.2 Primary Encryption……...24
7.3 Erasure Coding Technique to Split the File……..27
7.4 Re-encryption……29
7.5 File Transfer……..……32
7.6 File Retrieved From servers……..37
7.7 Decryption……41
7.8 Merge The Decrypted Files…..……44
7.9 Final Decryption……...46

Conclusion and Future Studies……..48
8.1 Conclusion……49
8.2 Future Studies……..……49

References……50



Introduction

Topics

- **Background**
- **Motivation**
- **Problem Statement**
- **Aims and Objectives**
- **Limitations of this Project**

1.1 Background

In this recent times, cloud storage systems have gained much popularity because of it's scalability, cost-effectiveness, and convenience. Users of these systems may access their data at any time and from any location by storing their data on different servers. Thoughts concerning data security and privacy have been raised as a result of the growing reliance on cloud storage. Vulnerabilities including unauthorized access, data loss, and data leakage hamper traditional methods of data storage on the cloud.

To address these concerns, Erasure coding and re-encryption of the data is a promising solution to ensure the CIA triad of the data. Erasure coding helps to split data into multiple segments and re-encryption technique is used to encrypt those splitted data and store them on multiple servers.

1.2 Motivation

Erasure coding offers fault tolerance and robustness but also it has some challenges in terms of providing data forwarding and security. Data forwarding describes the procedure of sending data from the client to the storage nodes in a typical cloud storage system.Without sufficient security measures, however, this procedure may be exposed to intrusions like eavesdropping, data manipulation, and man-in-the-middle attacks.

The goal of this project is to design and build a secure erasure code-based cloud storage system that not only assures data availability and durability but also contains secure data transfer protocols. By solving the security risks connected with data transmission, we hope to provide consumers with a trustworthy and dependable cloud storage option.

The proposed system will use advanced cryptographic algorithms, secure communication protocols and efficient erasure coding algorithms to meet the desired security goals. We also use re-encryption of the splitted data before sending them to servers to provide extra security of the data. By combining these components, we want to create a cloud storage system that provides both robust data security and efficient data forwarding, overcoming the weaknesses of present techniques and contributing to the growth of secure cloud storage technologies.

Overall, The combination of Erasure coding, Re-encryption, Secure data forwarding enhance the security and reliability of the cloud storage systems and it is the motivation of this project.

1.3 Problem Statement

The growing use of cloud storage systems has resulted in several advantages in terms of scalability, cost-effectiveness, and accessibility. However, these technologies frequently suffer serious security and privacy issues. Traditional cloud storage systems are at risk for data loss and illegal access, making sensitive user data vulnerable to breaches.

One of the most serious complexities in cloud storage systems is maintaining data availability and durability in the face of hardware failures or server outages. By storing data among various storage nodes and allowing the system to recreate lost or corrupted data, erasure codes have emerged as a powerful solution for solving this difficulty. When it comes to safe data transfer, erasure codes, on the other hand, provide their own set of issues.

The issue is that erasure code-based cloud storage systems lack safe data transmission techniques. The process of transferring data from the client to the storage nodes is referred to as data forwarding. This method is vulnerable to eavesdropping, data manipulation, and man-in-the-middle attacks if suitable security measures are not implemented.

As a result, the project's goal is to develop and construct a secure erasure code-based cloud storage system with flexible and efficient data forwarding methods. The system should secure data confidentiality and integrity during transmission, defend against unwanted access and manipulation, and prevent assaults that could harm the cloud storage system's security. By overcoming these obstacles, the project hopes to provide consumers with a reliable and secure cloud storage option that overcomes the limits of existing systems.

1.4 Aims and Objectives

The aim of this project is to enhance the privacy and security of cloud storage systems with Erasure coding Techniques and Re-encryption. Find out the vulnerabilities of the existing system and to overcome those vulnerabilities. Split data in different parts and provide re-encryption of those splitted parts to ensure extra security and store them on different servers.

The objective of this project is to analyze the existing system and find the vulnerabilities of that system. To design a secure erasure code based cloud system that ensures the confidentiality,

integrity, and authenticity of stored data. To develop a secure data forwarding mechanism to protect the stored data on the servers. To compare the proposed system with the existing system for finding out how the vulnerabilities of the existing system are being overcomed. To offer guidance and ideas for setting up and running secure data forwarding and erasure code-based cloud storage systems.

1.5 Limitations of this Project

The limitations of this project are:

- The project is mainly concerned with the security of data forwarding while it is being transmitted, but it does not address additional issues like access control, user authentication, or data privacy outside of data transmission.
- The project takes a reliable cloud storage environment for granted and ignores the difficulties brought on by unreliable or hostile cloud providers.
- The system will be implemented and evaluated in a controlled environment, therefore it's probable that some real-world network circumstances and performance fluctuations won't be taken into consideration.



Literature review

Topics

- **Cloud Storage System**
- **Erasure Coding Techniques**
- **Secure Data Forwarding Mechanisms**

2.1 Cloud Storage System

Cloud storage systems provide users with the ability to store and access their data remotely through a network of distributed servers. The users can store and retrieve data on the storage server rather than storing them on their local computer or system.

[1] Dr. Joseph Carl Robnett Licklider (1960) introduced the concept of Cloud Storage System. His concept paper describes the concept of cloud storage where users can store their data from different locations over the world.

[2] Michael luby (1998) Introduced Tornado codes, which enable efficient data repair and retrieval in distributed storage systems.

2.2 Erasure Coding Techniques

Erasure coding (EC) is a method of data protection in which data is broken into fragments, expanded and encoded with redundant data pieces and stored across a set of different locations or storage media.

[1] Robert G. Gallager (1960) introduced the concept of erasure coding.The contributions of Gallager have significantly influenced the advancement and adoption of erasure coding as a reliable and efficient method for data protection and recovery.

[2] Irving S. Reed and Gustave Solomon (1962) developed erasure coding techniques and mechanisms.

2.3 Secure Data Forwarding Mechanisms

Secure data forwarding mechanisms are essential to protect against attacks such as eavesdropping, data tampering, and man-in-the-middle attacks during the transmission process.

[1] Diffie and Hellman (1976) invented a data forwarding key exchange mechanism.

[2] Rivest, Shamir, and Adleman (1977) invented the RSA algorithm for secure data forwarding.



Existing System

Topics

- **General Overview of Existing System**
- **Disadvantages of the Existing System**

3.1 General Overview of Existing System

In Existing system a straightforward integration method is used. General encryption schemes protect data confidentiality, but also limit the functionality of the storage system because a few operations are supported over encrypted data. Storing data without doing erasure coding in a third party's cloud system causes serious concern on data confidentiality. In the existing system, constructing a secure storage system that supports multiple functions is challenging when the storage system is distributed and has no central authority.

3.2 Disadvantages of the Existing System

There are three problems in the above straightforward integration of encryption and encoding. The disadvantages are:

- The user has to do most computation and the communication traffic between the user and storage servers is high.
- Storing data on a third party's cloud system without using erasure coding causes serious data confidentiality.
- There is high possibility that the third party cloud can see the whole data as there is no erasure coding and re-encryption is used.

CHAPTER 4

Proposed System Methodology

Topics

- **Overview of the Proposed System**
- **System Architecture**
- **Data Flow Diagram**
- **Advantages of the Proposed System**
- **Components and Modules**

4.1 Overview of the Proposed System

In the proposed system we introduce a re-encryption scheme and integrate it with a decentralized erasure code such that a secure distributed storage system is formulated. By using the threshold proxy re-encryption scheme, we present a secure cloud storage system that provides secure data storage and secure data forwarding functionality.

Here on this system a user can store his data more secure way than the typical existing cloud storage system. In this method the data is encrypted before the erasure coding technique is used. After that on the encrypted data the erasure coding is used to split that data into many parts. After that the re-encryption technique is used on the splitted data parts to make more secure.

In the typical existing system the data is stored on one cloud server. But here the re-encrypted data is stored on different storage servers that makes it more secure than the typical existing system.

As the splitted re-encrypted data is stored on different storage servers so by compromising one server one one can find out the real data.

The main technical contribution is that the proxy re-encryption scheme supports encoding operations over encrypted messages as well as forwarding operations over encoded and encrypted messages. Our method fully integrates encrypting, encoding, and forwarding.

We propose a new threshold proxy re-encryption scheme and integrate it with a secure decentralized code to form a secure distributed storage system. The encryption scheme supports encoding operations over encrypted messages and forwarding operations over encrypted and encoded messages. The tight integration of encoding, encryption, and forwarding makes the storage system efficiently meet the requirements of data robustness, data confidentiality, and data forwarding. Accomplishing the integration with consideration of a distributed structure is challenging. Our system meets the requirements that storage servers independently perform encoding and re-encryption and key servers independently perform partial decryption. Moreover, we consider the system in a more general setting than previous works. This setting allows more flexible adjustment between the number of storage servers and robustness.

4.2 System Architecture

The Proposed system architecture is given below:

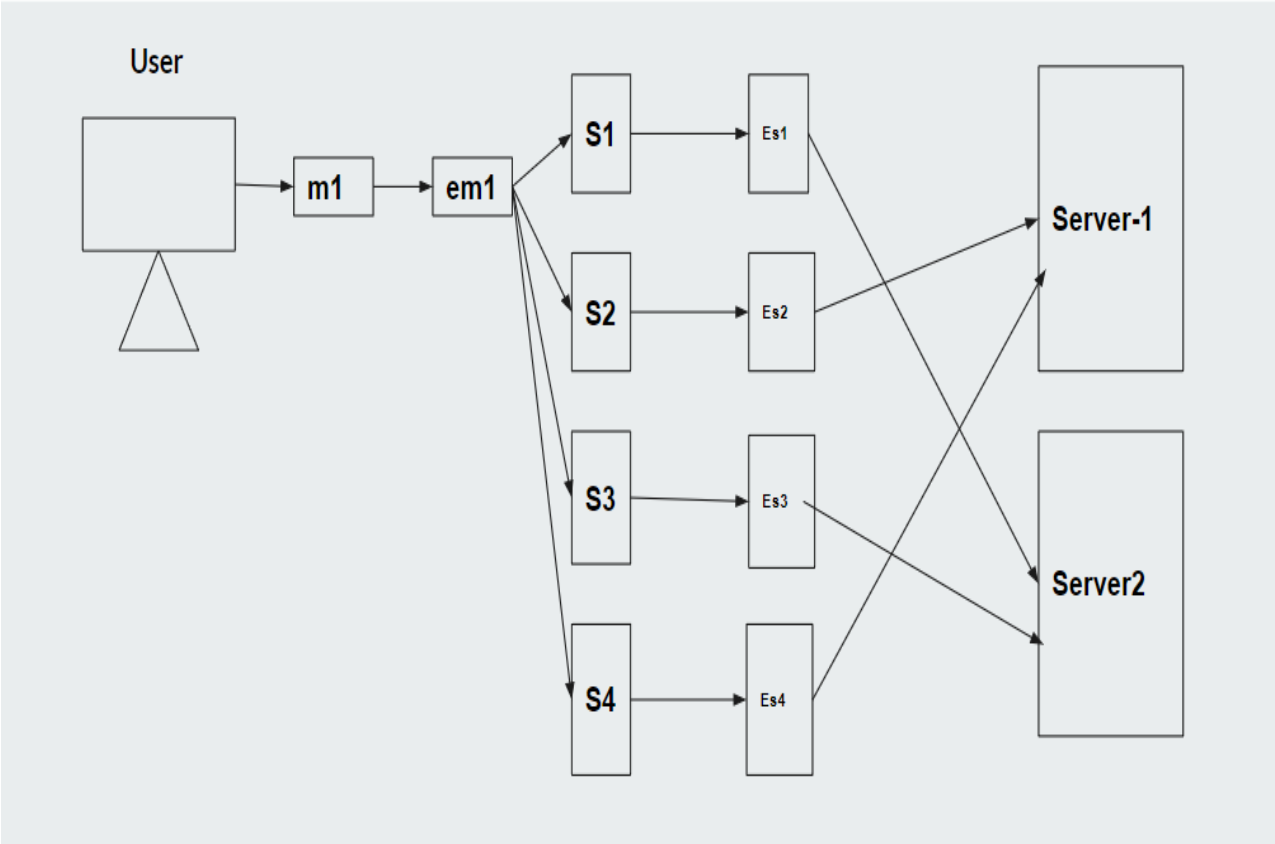


Fig 4.2:-Proposed System Architecture

Here from this architecture above we can see our proposed system. Here a user wants to store a message or data that is m1 on the server. First level encryption is done with a key and we get em1. After that erasure coding is used and the encrypted data is splitted into 4 parts those are S1, S2, S3 and S4. After that Re-encryption scheme is used to re-encrypt those splitted data with a key and we get Es1, Es2, Es3 and Es4. Now those re-encrypted splitted data are stored on 2 different cloud storage servers. Here we can see Es2 and Es4 are stored on Server-1, Es1 and Es3 are stored on Server-2.

Now after the stroing part is done if the user wants to retrieve his data he can call it on his system and this system will perform reverse action and send his data to his machine.

The reverse action is the Es1, Es2, Es3, Es4 are decrypted using the key and S1, S2, S3, S4 are formed. After that S1, S2, S3 and S4 are marged and em1 is formed. The the primary level decryption is done with the key and the user get is original data.

4.3 Data Flow Diagram

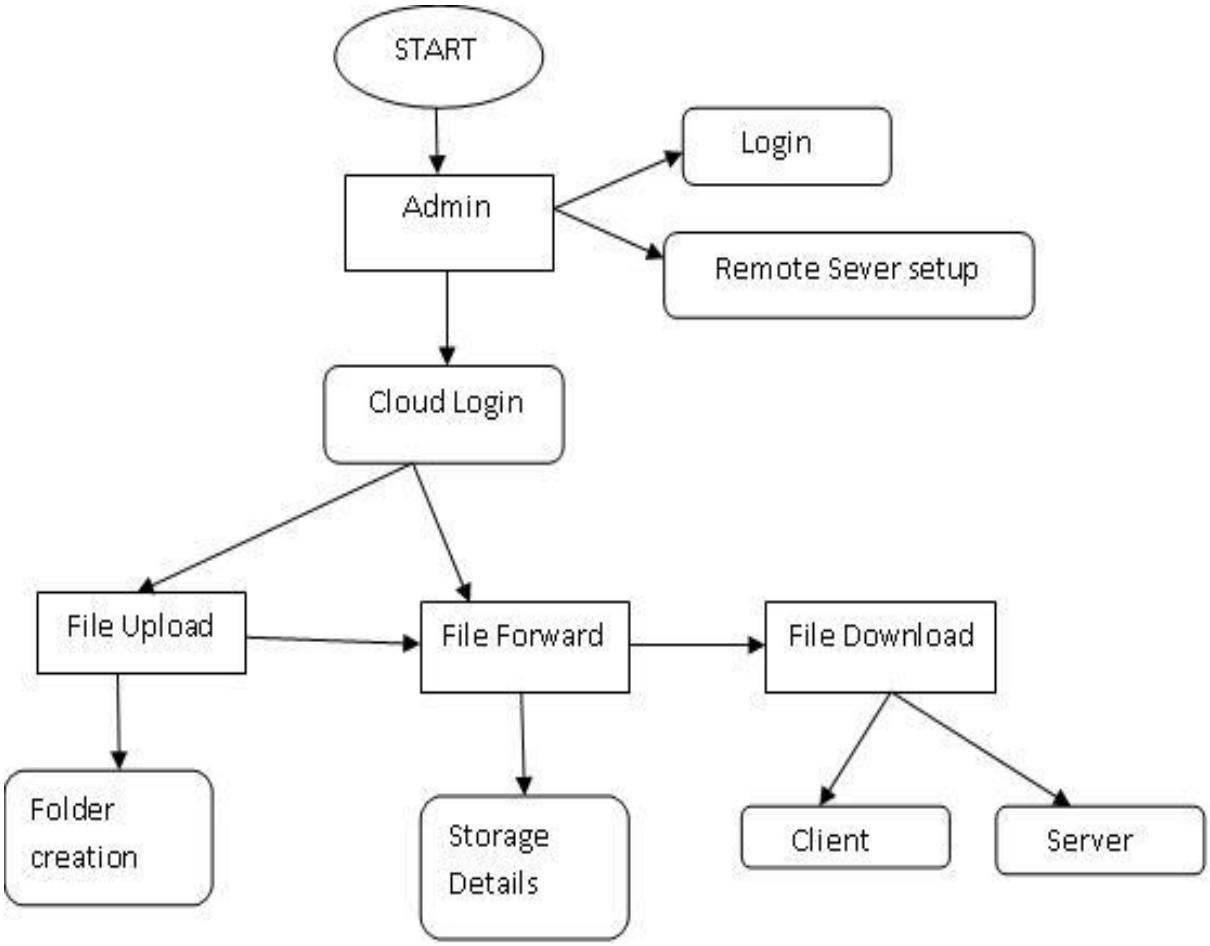


Fig 4.3:-Data Flow Diagram

4.4 Advantages of the Proposed System

The advantages of the proposed System are given below:

- Primary level encryption is done to the data
- Erasure coding is used to split the data in different parts so that those splitted data can be stored on different servers.
- The erasure coding provides extra security.
- Re-encryption scheme is used to re-encrypted those splitted data.
- Tight integration encoding, encryption, forwarding makes the storage system efficiently meet the requirements of data robustness, data confidentiality and data forwarding.
- More flexible adjustment between the storage servers.

4.5 Components and Modules

In this section we will discuss about the components and module used to construct this proposed system.

4.5.1 Cloud Data Storage Module

In this system we will use two cloud storage server to store data. The two storage server module will perform these operations:

- Form the distributed storage system in the cloud.
- Store data fragments generated by erasure coding.
- Implement data redundancy and fault tolerance mechanisms.
- Participate in data reconstruction and recovery processes.

4.5.2 Data Encryption Module

In this module data is encrypted on the primary stage with a key. The operations performed by this module is given below:

- Data is encrypted on the primary level.
- An integer key is used to encrypt this data.

4.5.3 Erasure-coding module

The operations performed by Erasure coding module is given below:

- Performs encoding and decoding operations on data to ensure fault tolerance and data recovery.
- Distributes data fragments across multiple storage nodes using erasure coding techniques.
- Implements efficient data reconstruction algorithms to recover lost or corrupted data fragments.

4.5.4 Re-encryption Module

The operations performed by re-encryption module is given below:

- Re-encrypt the splitted data fragments.
- It uses a key to re-encrypt those splitted fragments.
- Stores that key for future decryption.
- Provides extra security by Re-encrypting those splitted fragments.

4.5.6 Data forwarding and retrieving module

The operations performed by Data forwarding and retrieving module is given below:

- Ensures the confidentiality, integrity, and authenticity of data during transmission.
- Implements secure communication protocols, such as TLS (Transport Layer Security), to establish secure channels between the client and storage nodes.
- Protects against different kinds of cyber attack.
- Can securely forward data to others and retrieve data into the original form.

These are the components and modules used in this proposed system.



Requirements

Topics

- **Requirements**
- **Hardware requirements**
- **Software Requirements**

5.1 Requirements

To implement this proposed system there are two kinds of requirements. These two requirements must be fulfilled to implement this system.

5.1.1 Hardware requirements

Minimum Hardware requirements to implement this proposed system is given below:

- Laptop or Desktop
- RAM - 8 GB
- Hard disk - 256 GB
- Keyboard - Standard Windows Keyboard
- Processor - Corei5

5.1.2 Software Requirements

Software requirements to implement this proposed system is given below:

- Operating system - Windows 10/11
- Virtual machine - Vmware or Virtual box
- Virtual Server - Ubuntu or any other virtual server
- Script - Python and bash

These Hardware and Software requirements must be fulfilled to implement this project.



Implementation

Topics

- **Bash script**
- **Primary Encryption**
- **Erasure Coding Technique to Split the File**
- **Re-encryption**
- **File Transfer**
- **File Retrieved From servers**
- **Decryption**
- **Merge The Decrypted Files**
- **Final Decryption**

6.1 Bash script

To implement this project i have wrote a bash script that will do the full automation of this project and the required python scripts to run this project are integrated with this bash script. The bash script name is script7.sh and it is set up on the kali linux machine as the kali linux is the user machine for this project.

The bash script script7.sh is given below:

```
  GNU nano 6.3                                                    script7.sh
#!/bin/bash

# Function to execute primary encryption script
execute_primary_encryption_decryption() {
    python /root/Desktop/Finalyear/hello.py
}

# Function to execute split the file script
execute_split_file() {
    read -p "Enter the file to split: " file_to_split
    read -p "Enter the size per file (e.g., 1K, 1M, 1G): " size_per_file

    split -b "$size_per_file" "$file_to_split" "output_prefix"

    echo "File split completed."
}

# Function to execute re-encryption script
execute_re_encryption() {
    python /root/Desktop/Finalyear/encryptioncode.py
}

# Function to execute file transfer to servers script
execute_file_transfer() {
    python /root/Desktop/Finalyear/trantwoserver.py
}

# Function to execute receive files from servers script
execute_receive_files() {
    python /root/Desktop/Finalyear/rectwoserver.py
}

# Function to execute final decryption script
execute_final_decryption() {
    python /root/Desktop/Finalyear/dec1.py
}

# Function to execute merge script
execute_merge() {
    read -p "Enter the path and name of the merged file: " merged_file_path

    while true; do
        read -p "Enter the path of the split file to merge (or 'done' to finish): " split_file_path

        if [ "$split_file_path" = "done" ]; then
            break
```

```
    fi

    cat "$split_file_path"* >> "$merged_file_path"
done
    echo "Files merged successfully."
}

# Prompt the user to choose an option
while true; do
    echo "Select an option:"
    echo "1. Primary Encryption and Final Decryption"
    echo "2. Split the File"
    echo "3. Re-encryption"
    echo "4. File Transfer to the Servers"
    echo "5. Receive Files from the Servers"
    echo "6. Decryption"
    echo "7. Merge"
    echo "8. Exit"
    read -p "Enter your choice: " choice

    case $choice in
        1)
            execute_primary_encryption_decryption
            ;;
        2)
            execute_split_file
            ;;
        3)
            execute_re_encryption
            ;;
        4)
            execute_file_transfer
            ;;
        5)
            execute_receive_files
            ;;
        6)
            execute_final_decryption
            ;;
        7)
            execute_merge
            ;;
        8)
            echo "Exiting..."
            exit 0
            ;;
        *)
            echo "Invalid choice. Please try again."
            ;;
    esac

    echo # Add a line break for readability
    echo # Add a line break for readability
    echo "Press Enter to continue..."
    read # Wait for Enter key press before displaying options again
    echo # Add a line break for readability
done
```

Fig 6.1:-Bash script

From the above screenshot we can see the full bash script to automate this project

6.2 Primary Encryption

The first step of this project is to encrypt a file on the primary level. We have to run the bash script on the kali linux user machine with **./script7.sh** command. After running the bash script we can see the overview with 8 options on the kali linux terminal.

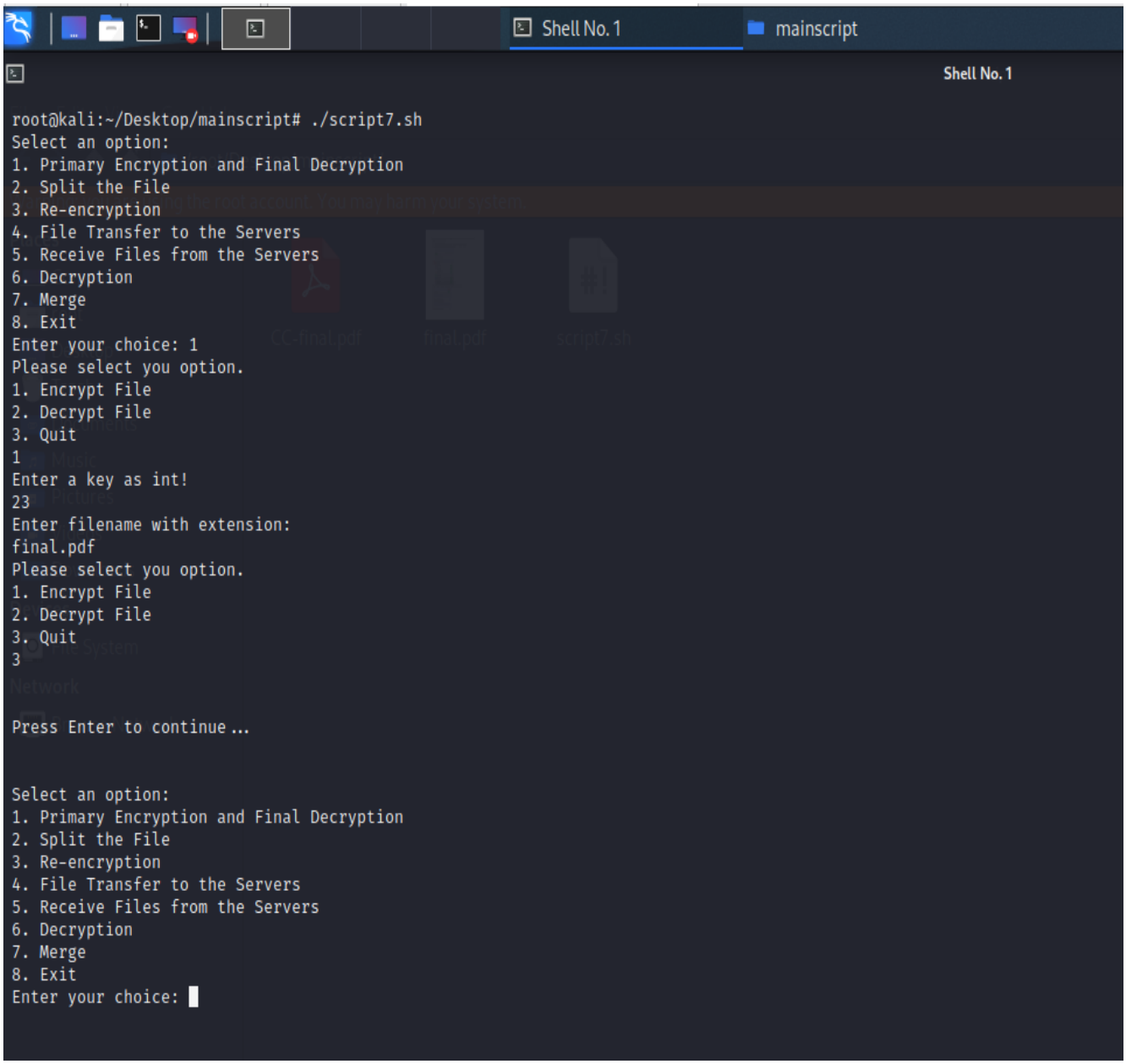


Fig 6.2.1:-Primary Encryption

From the Figure 7.2 we can see the overview of the bash script that will come with 8 options. The options are:

1. Primary Encryption and Final Decryption
2. Split the File
3. Re-encryption
4. File Transfer to the Servers
5. Receive Files from the Servers
6. Decryption
7. Merge
8. Exit

Then i have to enter choice of which operation i want to perform by selecting from those 8 options.

To encrypt a file on primary level we have to enter option 1. Then it will ask us to enter our option to encrypt the file. Here we have to select option 1 to encrypt the file. After that it will ask us to enter a key in integer format to encrypt the file. Here we enter the key as 23. Then it will ask us to enter the file with extension to encrypt the file. Here we enter the filename that is **final.pdf**

Then we have to select option 3 Quit option to come out from the loop and we will get those 8 options again.

After Encrypting the final.pdf file we will get a encrypted file that is CC-final.pdf

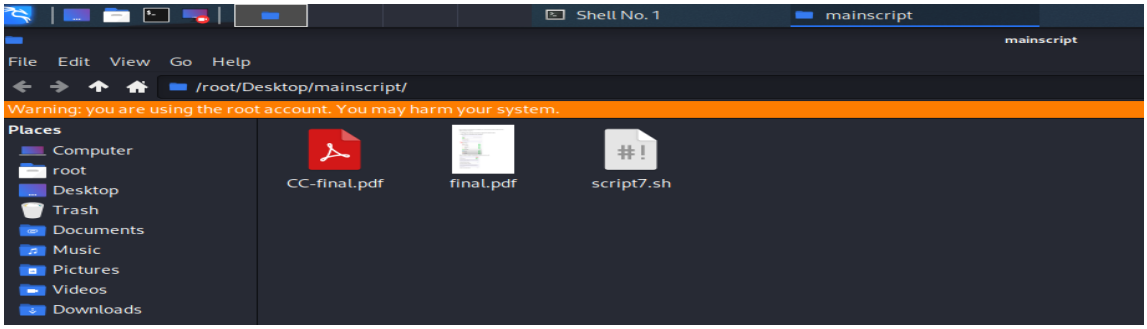


Fig 6.2.2:-Encrypted file

The python script behind this Primary encryption is given below:

```
def Encrypt(filename, key):
    file = open(filename, "rb")
    data = file.read()
    file.close()

    data = bytearray(data)
    for index, value in enumerate(data):
        data[index] = value ^ key


    file = open("CC-" + filename, "wb")
    file.write(data)
    file.close()

def Decrypt(filename, key):
    file = open(filename, "rb")
    data = file.read()
    file.close()

    data = bytearray(data)
    for index, value in enumerate(data):
        data[index] = value ^ key


    file = open(filename, "wb")
    file.write(data)
    file.close()

choice = ""
while choice != "3":
    print("Please select you option.")
    print("1. Encrypt File")
    print("2. Decrypt File")
    print("3. Quit")
    choice = input()
    if choice == "1" or choice == "2":
        key = int(input("Enter a key as int!\n"))
        filename = input("Enter filename with extension:\n")
    if choice == "1":
        Encrypt(filename, key)
    if choice == "2":
        Decrypt(filename, key)
```

Fig 6.2.3:-Python script to Encrypt a file

This python script is used to encrypt the final.pdf file and this script is integrated with the bash script on the execute_primary_encryption_decryption() function. From the above 7.1 Bash script figure we can see that we have defined the path of the Encryption python script which is /root/Desktop/Finalyear/hello.py so the name of the python script is hello.py

6.3 Erasure Coding Technique to Split the File

The second step is to split the Encrypted CC-final.pdf file in multiple files. This process is known as Erasure coding.

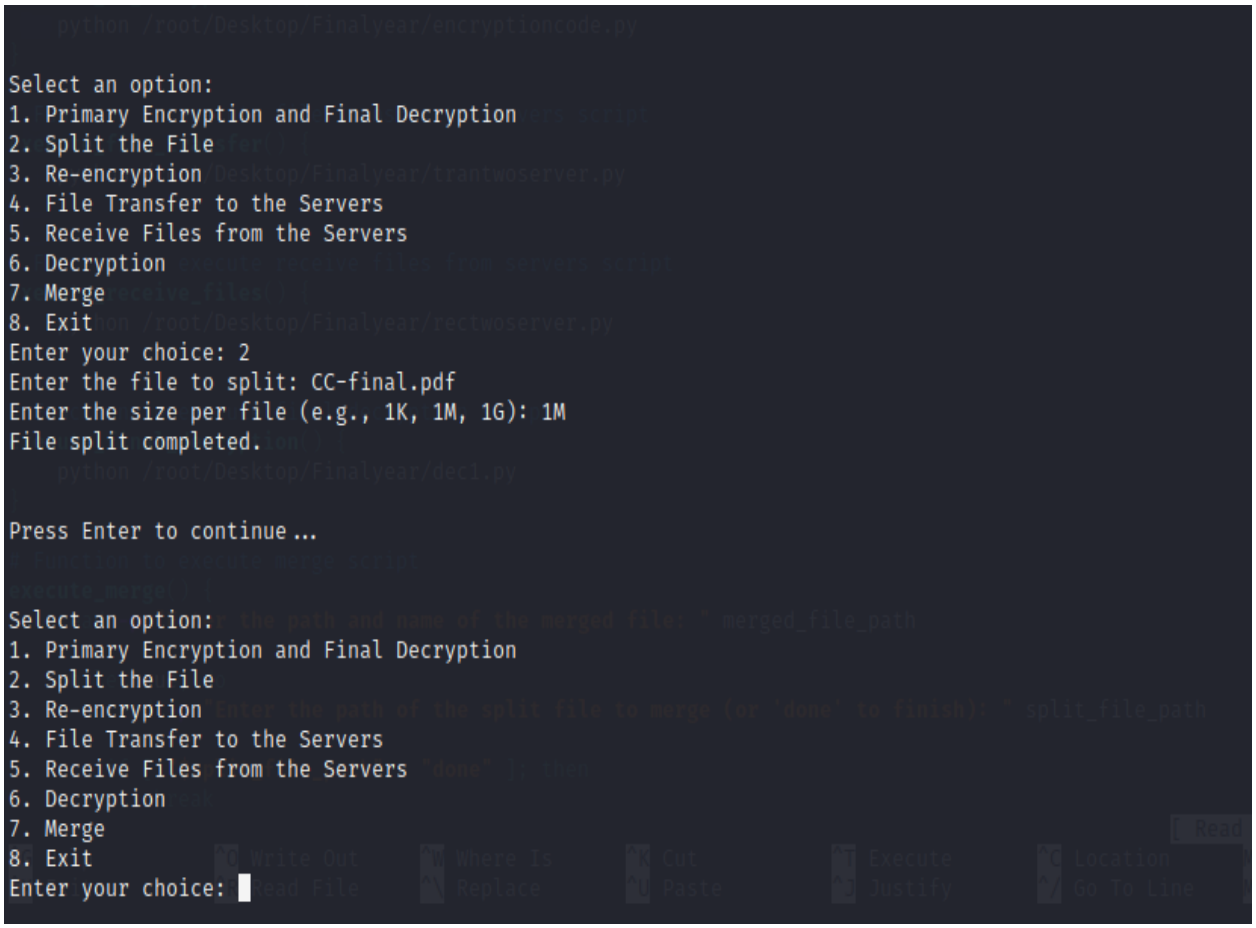


Fig 6.3.1:-Split the File

To split the encrypted CC-final.pdf file in multiple sections we have to select option 2 that is Split the File from the Bash script script7.sh . After that it will ask us to Enter the file that we want to split. We have to give here the encrypted file that is CC-final.pdf. Then after that it will ask us to specify the size of every splitted file. We can specify the splitted file size by Kilobyte, Megabyte,

Gigabyte segments. As the CC-final.pdf is 2.6 Megabytes so we have to enter 1M here that will split the file into three parts each part containing 1 megabyte. Then we can see the file split completed.

After that we will get 3 splitted files those are output_prefixaa, output_prefixab, output_prefixac.

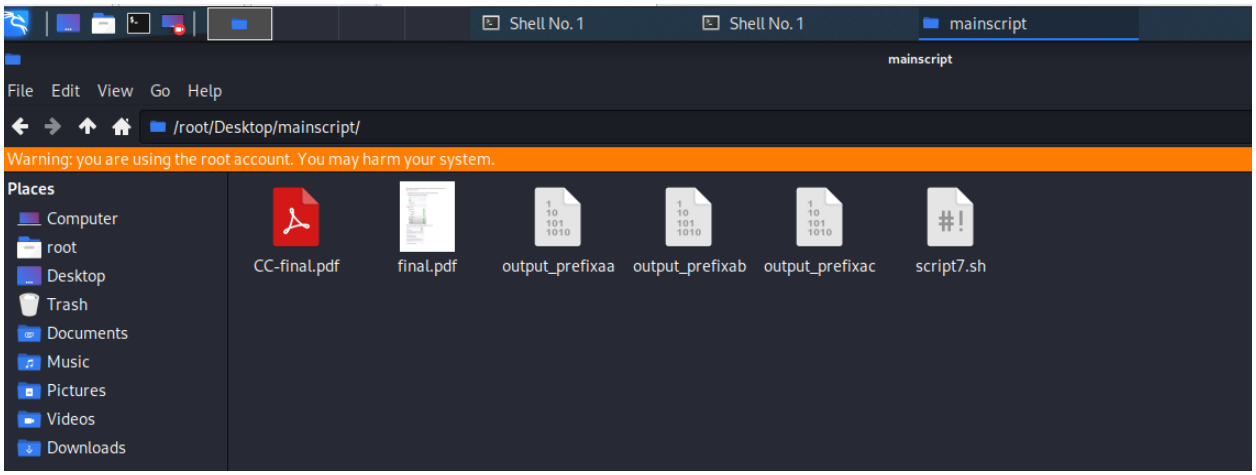


Fig 6.3.2:-Splitted Files

From the above Figure we can see the 3 splitted files. Here to split the file in multiple files we used a kali linux tool called split and integrate that tool with the bash script script7.sh on the execute_split_file() function.

```
# Function to execute split the file script
execute_split_file() {
    read -p "Enter the file to split: " file_to_split
    read -p "Enter the size per file (e.g., 1K, 1M, 1G): " size_per_file

    split -b "$size_per_file" "$file_to_split" "output_prefix"

    echo "File split completed."
}
```

Fig 6.3.3:-Split script

As we can see from the above Figure this script will split a file into multiple files according to the specified size the we enter on the bash script.

6.4 Re-encryption

The third step is to re-encrypt those 3 splitted files to give extra security.

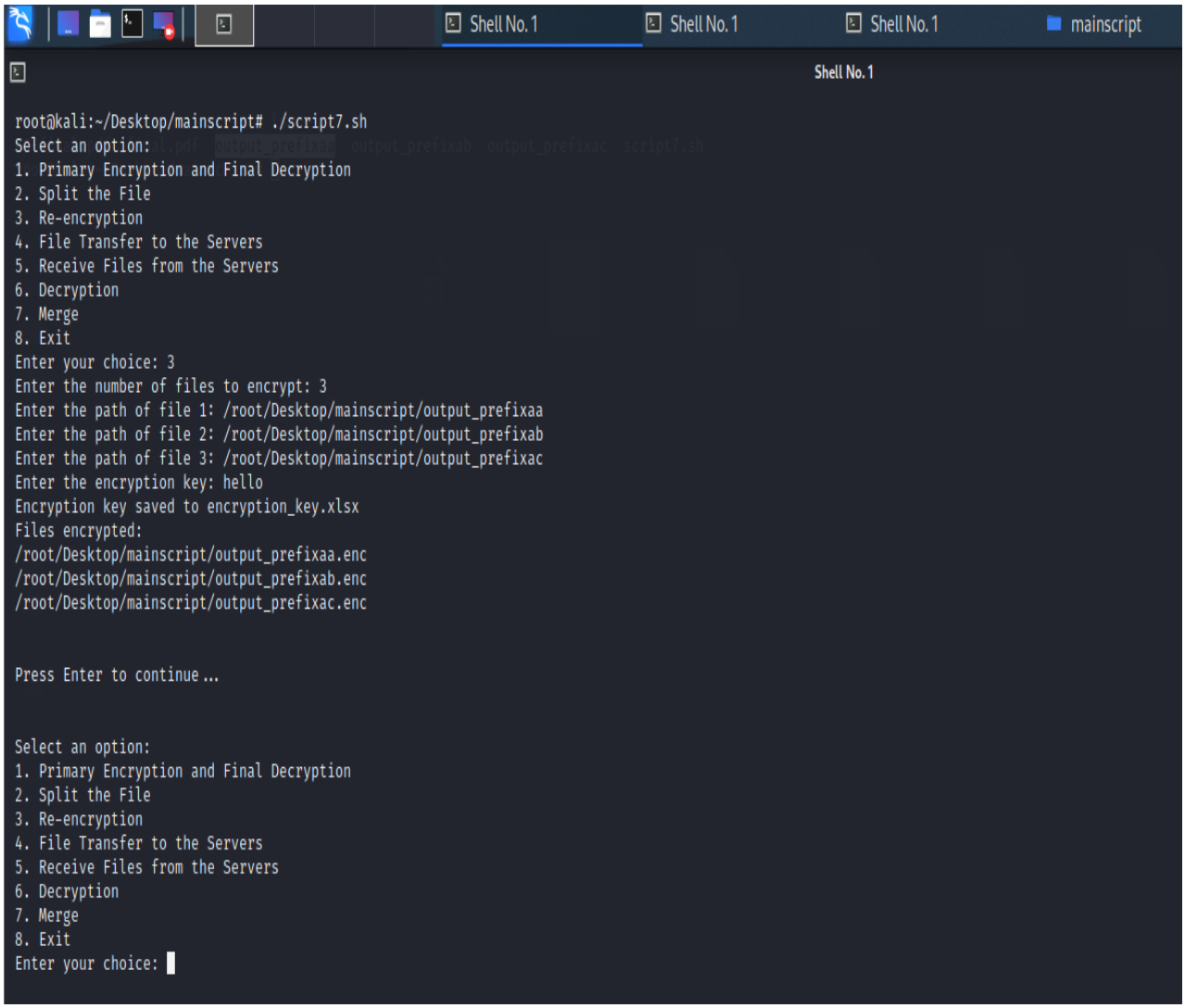


Fig 6.4.1:-Re-encryption

To re-encrypt those 3 splitted file we have to enter option 3 from the bash script that is Re-encryption. Then we have to select the number of files that we want to re-encrypt. Here as we

want to re-encrypt 3 files that's why we have to enter 3 . Then We have to enter the path of those splitted file to re-encrypt them.

The path of those 3 splitted files are:

- /root/Desktop/mainscript/output_prefixaa
- /root/Desktop/mainscript/output_prefixab
- /root/Desktop/mainscript/output_prefixac

Then we have to enter the encryption key that will be used to re-encrypt those files. Here we use the key **hello.**

After that we can see the encryption key is saved to encryption_key.xlsx file. Then we can see the files are encrypted with the names of output_prefixaa.enc, output_prefixab.enc, output_prefixac.enc.

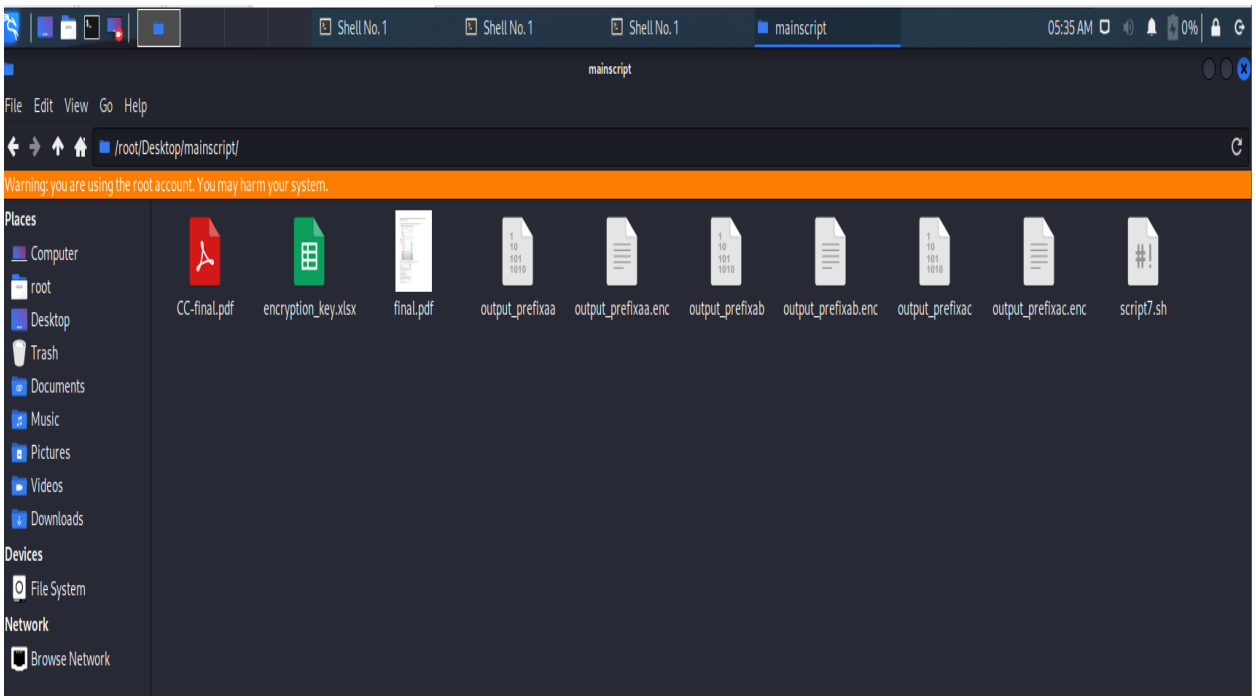


Fig 6.4.2:-Re-encrypted files and key file

For this Re-encryption scheme we have used a python script. The function of that python script is to re-encrypt those splitted files with the provided key and store that key for future decryption process.

The python script behind this Re-encryption:

```
import os
import hashlib
import base64
import openpyxl
from openpyxl import Workbook
from cryptography.fernet import Fernet

def encrypt_file(filename, key):
    output_filename = filename + '.enc'

    with open(filename, 'rb') as file:
        data = file.read()

    cipher_suite = Fernet(key)
    encrypted_data = cipher_suite.encrypt(data)

    with open(output_filename, 'wb') as output_file:
        output_file.write(encrypted_data)

    return output_filename
def save_key_to_excel(key):
    workbook = Workbook()
    sheet = workbook.active
    sheet.title = 'Encryption Key'
    sheet['A1'] = 'Key'
    sheet['B1'] = key.decode()

    excel_filename = 'encryption_key.xlsx'
    workbook.save(excel_filename)
    return excel_filename
# Get input files
filepaths = []
num_files = int(input("Enter the number of files to encrypt: "))

for i in range(num_files):
    filepath = input(f"Enter the path of file {i+1}: ")
    filepaths.append(filepath)

# Generate encryption key
password = input("Enter the encryption key: ").encode()
key = base64.urlsafe_b64encode(hashlib.sha256(password).digest())

# Encrypt files
encrypted_files = []
for filepath in filepaths:
    encrypted_file = encrypt_file(filepath, key)
```

```
encrypted_files.append(encrypted_file)

# Save encryption key to Excel file
excel_filename = save_key_to_excel(key)
print(f"Encryption key saved to {excel_filename}")

print("Files encrypted:")
for encrypted_file in encrypted_files:
    print(encrypted_file)
```

Fig 6.4.3:-Python script for Re-encryption

From the 7.1 figure we can see that this python script is integrated with the execute_re_encryption() function of the script7.sh bash script and we have defined the path of the script that is /root/Desktop/Finalyear/encryptioncode.py so the re-encryption script name is encryptioncode.py

6.5 File Transfer

The fourth step is to transfer those 3 re-encrypted files to the servers. Here we have taken two servers those are server1 and server2.

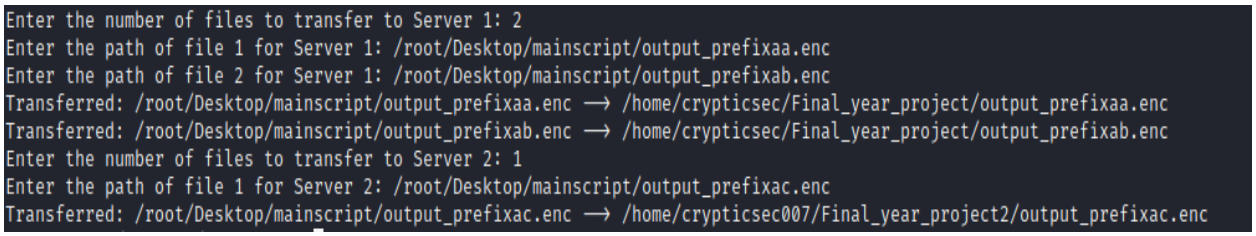


Fig 6.5.1:-File Transfer

Here to transfer those re-encrypted files we have to enter options 4 from the bash script that is File Transfer to the Servers. Then we have to select how many files we want to send to the Server1. Here we want to send 2 files to server1. After that we have to select the path of those two re-encrypted files that we want to send to the server1.

The path of those two files are:

- /root/Desktop/mainscript/output_prefixaa.enc
- /root/Desktop/mainscript/output_prefixab.enc

After that we have to select how many files we want to send to server2. Here we select 1. The we have to enter the path of that file.

- /root/Desktop/mainscript/output_prefixac.enc

From the above Figure we can see the files are transferred to the specified directory of those servers.

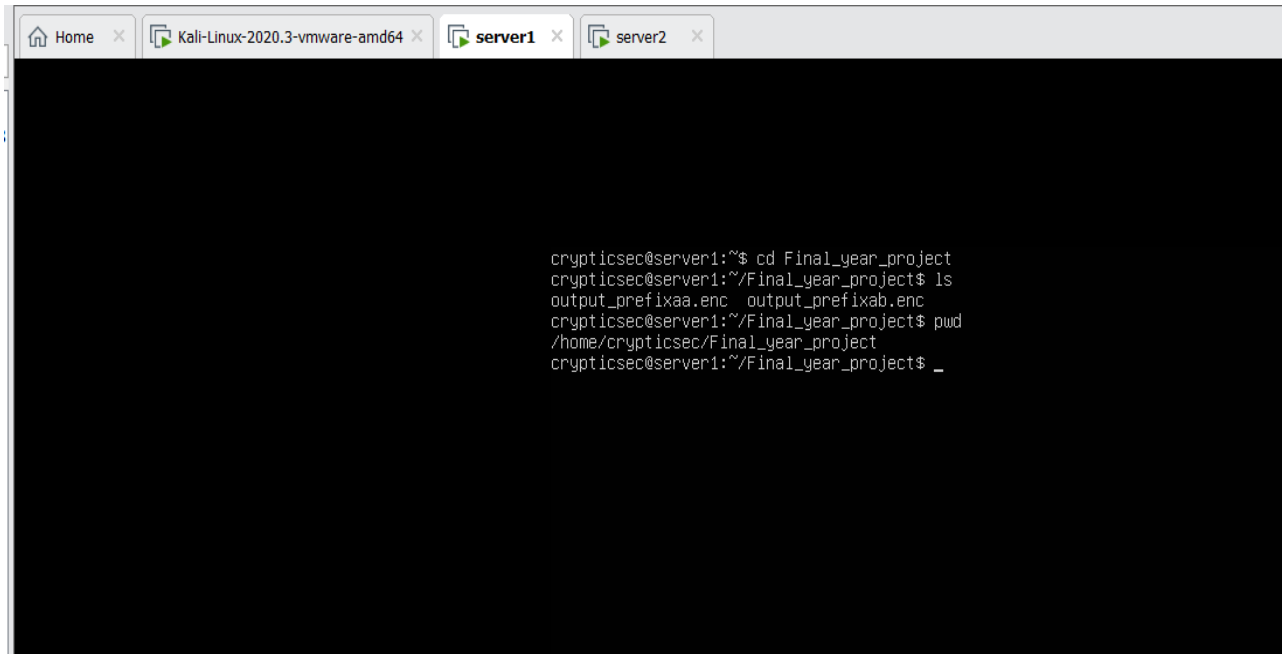


Fig 6.5.2:-Stored files on server1

From the above figure we can see that the first two files those are output_prefixaa.enc and output_prefixab.enc are successfully transferred and stored on server1 at the /home/crypticsec/Final_year_project directory.

Now we have to check if our third file successfully transferred on the server2.

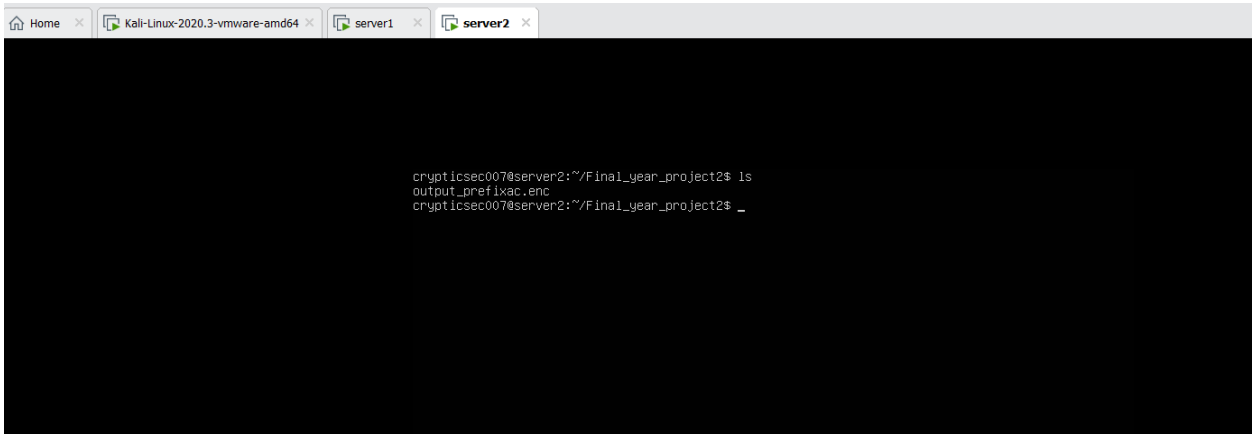


Fig 6.5.3:-Stored File on server2

Here from the above figure we can see that the third file that is output_prefixac.enc is successfully stored on the server2 at the /home/crypticsec007/Final_year_project2 directory.

The python script behind this file transfer name is trantwoserver.py which is integrated with the execute_file_transfer() function and the path of the python script is defined here on the bash script that is /root/Desktop/Finalyear/trantwoserver.py

Python script:

```
import os
import paramiko

def transfer_files(source_files, destination_dir, hostname, username, password):
    # Connect to the server
    client = paramiko.SSHClient()
    client.load_system_host_keys()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(hostname, username=username, password=password)

    # Create the destination directory on the server if it doesn't exist
    command = f'mkdir -p {destination_dir}'
    client.exec_command(command)

    # Transfer each specified file
    for source_file in source_files:
        # Extract the file name from the full path
        filename = os.path.basename(source_file)

        # Prepare the local and remote file paths
        local_path = source_file
        remote_path = os.path.join(destination_dir, filename)

        # Create a new SFTP client for file transfer
        sftp = client.open_sftp()
        sftp.put(local_path, remote_path)
        sftp.close()

        print(f"Transferred: {source_file} --> {remote_path}")

    # Close the SSH connection
    client.close()

# Prompt for the number of files and their paths for Server 1
num_files_server1 = int(input("Enter the number of files to transfer to Server 1: "))

source_files_server1 = []
for i in range(num_files_server1):
    filename = input(f"Enter the path of file {i+1} for Server 1: ")
    source_files_server1.append(filename)

destination_directory_server1 = '/home/crypticsec/Final_year_project'
```

```
hostname_server1 = '192.168.152.129'
username_server1 = 'crypticsec'
password_server1 = 'alif007'

transfer_files(source_files_server1, destination_directory_server1, hostname_server1, username_server1,
password_server1)

# Prompt for the number of files and their paths for Server 2
num_files_server2 = int(input("Enter the number of files to transfer to Server 2: "))

source_files_server2 = []
for i in range(num_files_server2):
    filename = input(f"Enter the path of file {i+1} for Server 2: ")
    source_files_server2.append(filename)

destination_directory_server2 = '/home/crypticsec007/Final_year_project2'
hostname_server2 = '192.168.152.130'
username_server2 = 'crypticsec007'
password_server2 = 'alif007'

transfer_files(source_files_server2, destination_directory_server2, hostname_server2, username_server2,
password_server2)
```

Fig 6.5.4: Python script to file transfer to the server

Here from the above script we can see that we have species the server1 and server2 ip address, username and password to send files to the servers. We have also defined the destination directory of the both server 1 and server 2 for storing files.

6.6 File Retrieved From servers

The fifth step is to retrieve those 3 files from the servers. To do this we have to select option 5 from the bash script that is Receive Files from the Servers.

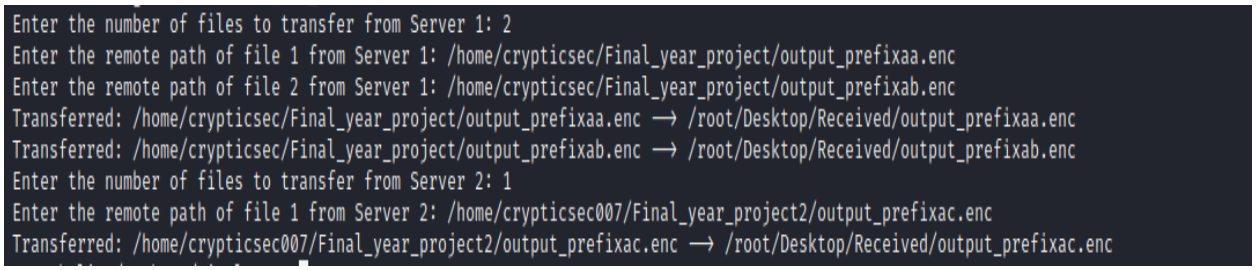


Fig 6.6.1:-File Retrieved from Servers

After selecting the option 3 from the bash script we have to the number of files that we want to retrieve from the server1. Here we want to retrieve 2 files from the server1. SO we have to enter 2 and after that we have to enter the path of those 2 files that are stored on server1.

The remote path of those two file on the server1 are:

- /home/crypticsec/Final_year_project/output_prefixaa.enc
- /home/crypticsec/Final_year_project/output_prefixab.enc

Then we have to select how many files we want to retrieve from server2. We have to select 1 here and then we have to enter the path of that file. That is:

- /home/crypticsec007/Final_year_project2/output_prefixac.enc

We can see those three files are successfully retrieved from the two servers and stored on /root/Desktop/Received/ directory.

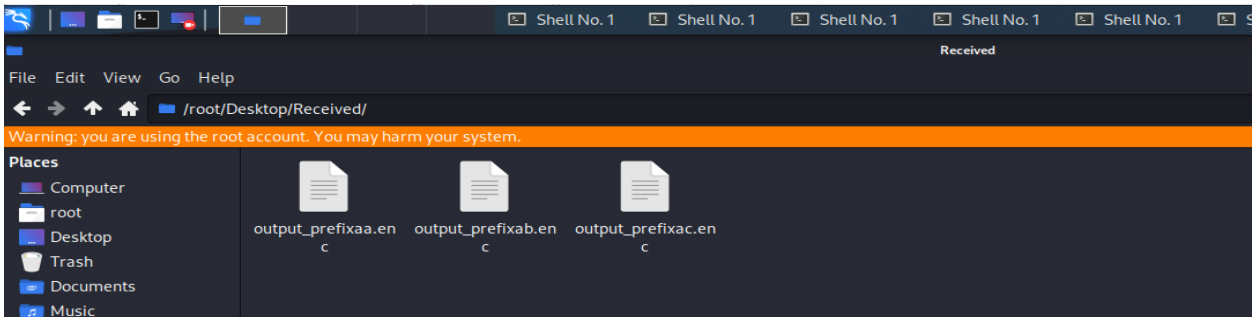


Fig 6.6.2:-File stored

From the above figure we can see that those three files are retrieved in this directory with the name output_prefixaa.enc, output_prefixab.enc, output_prefixac.enc.

There is a python script behind this retrieval and that python script is integrated with the bash file on the execute_receive_files() function and we have defined the path of the python script on the bash file that is /root/Desktop/Finalyear/rectwoserver.py so the python script name is rectwoserver.py

The python script behind this:

```
import os
import paramiko

def transfer_files(source_files, destination_dir, hostname, username, password):
    # Create the destination directory if it doesn't exist
    os.makedirs(destination_dir, exist_ok=True)

    # Transfer each specified file
    for source_file in source_files:
        # Extract the file name from the full path
        _, filename = os.path.split(source_file)

        # Prepare the local and remote file paths
        remote_path = source_file
        local_path = os.path.join(destination_dir, filename)

        # Connect to the server
        client = paramiko.SSHClient()
        client.load_system_host_keys()
        client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
        client.connect(hostname, username=username, password=password)

        # Create a new SFTP client for file transfer
        sftp = client.open_sftp()
        sftp.get(remote_path, local_path)
        sftp.close()

        print(f"Transferred: {source_file} --> {local_path}")

        # Close the SSH connection
        client.close()

# Prompt for the number of files and their paths from Server 1
num_files_server1 = int(input("Enter the number of files to transfer from Server 1: "))

source_files_server1 = []
for i in range(num_files_server1):
    filename = input(f"Enter the remote path of file {i+1} from Server 1: ")
    source_files_server1.append(filename)

destination_directory_server1 = '/root/Desktop/Received'
```

```
hostname_server1 = '192.168.152.129'
username_server1 = 'crypticse'
 password_server1 = 'alif007'
transfer_files(source_files_server1, destination_directory_server1, hostname_server1, username_server1,
password_server1)

# Prompt for the number of files and their paths from Server 2
num_files_server2 = int(input("Enter the number of files to transfer from Server 2: "))

source_files_server2 = []
for i in range(num_files_server2):
    filename = input(f"Enter the remote path of file {i+1} from Server 2: ")
    source_files_server2.append(filename)

destination_directory_server2 = '/root/Desktop/Received'
hostname_server2 = '192.168.152.130'
username_server2 = 'crypticsec007'
password_server2 = 'alif007'

transfer_files(source_files_server2, destination_directory_server2, hostname_server2, username_server2,
password_server2)
```

Fig 6.6.3:- Retrieval python script

Here from the above python script we can see that we have defined here the server1 and server2 ip address, username, password to retrieve files from those two servers. We have also defined the destination directory of the files to store and that is /root/Desktop/Received.

6.7 Decryption

The sixth step is to Decrypt the re-encrypted files.

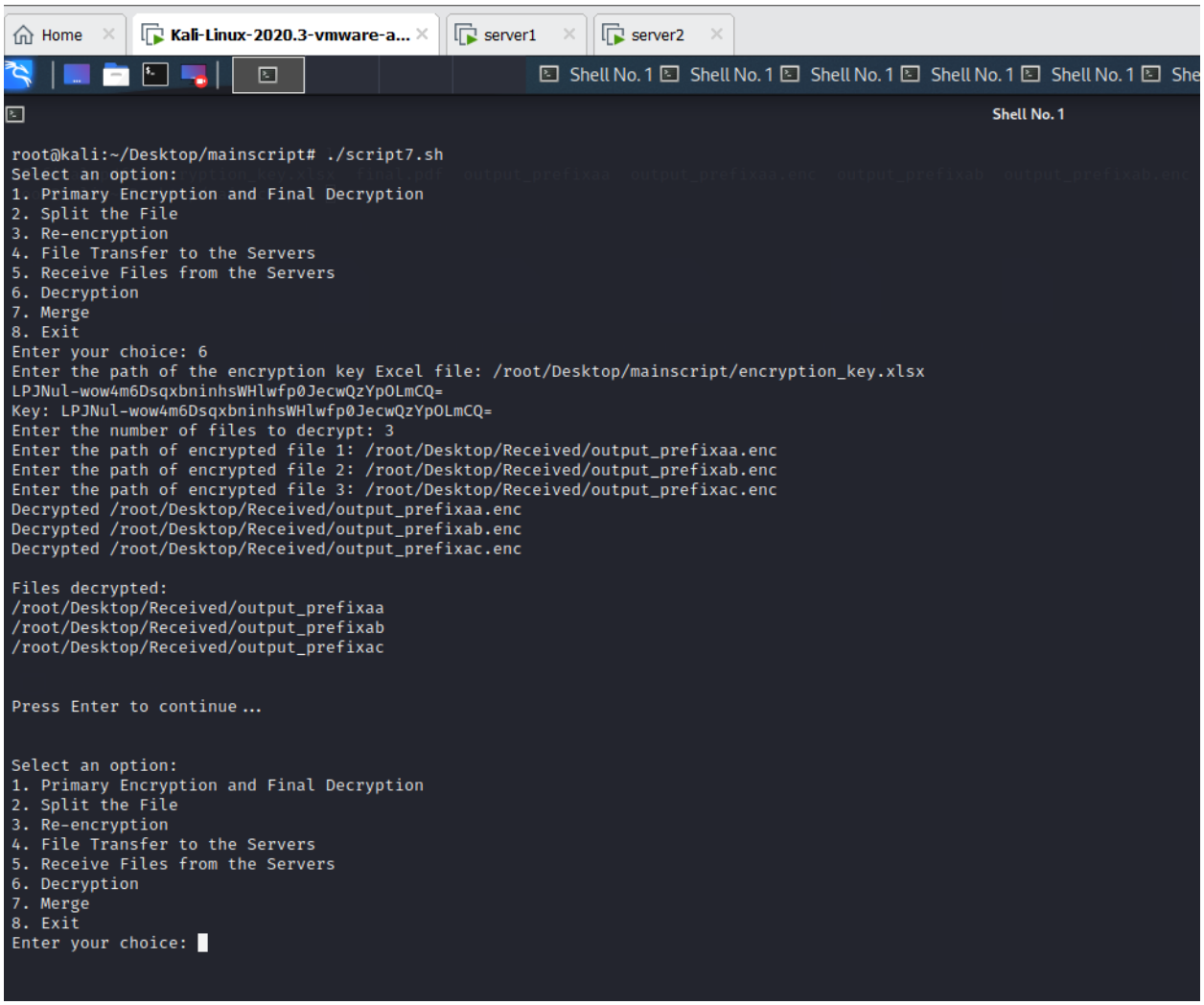


Fig 6.7.1:-Decryption

To do this we have to enter option 6 from the bash script that is Decryption. Then we have to enter the path of the key file that was created while re-encryption process. The path is /root/Desktop/mainscript/encryption_key.xlsx
Then we have to enter the number of files we want to decrypt. Here we want to decrypt 3 files so we have to enter 3. Then we have to enter the path of the 3 files that we want to decrypt.

The path of the files are:

- /root/Desktop/Received/output_prefixaa.enc
- /root/Desktop/Received/output_prefixab.enc
- /root/Desktop/Received/output_prefixac.enc

Then we can see the files are decrypted on the /root/Desktop/Received directory.

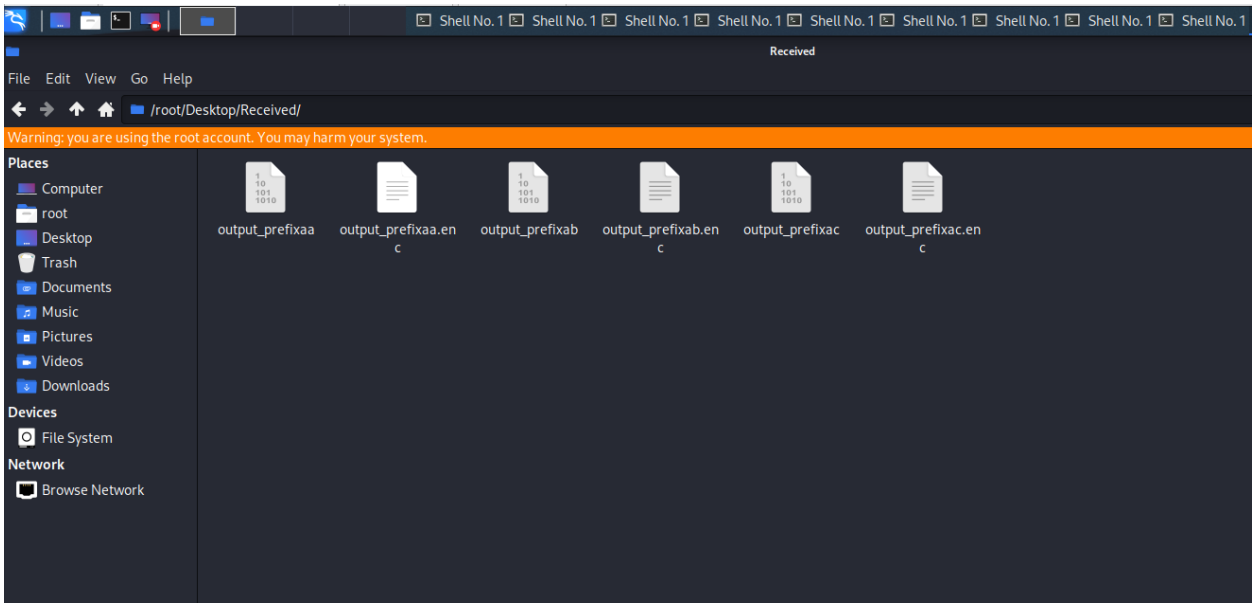


Fig 6.7.2:-Decrypted files stored

We can see the decrypted files are output_prefixaa, output_prefixab, output_prefixac

The python script behind this decryption is dec1.py and that script is integrated with the bash script in execute_final_decryption() function. The path of the python script is defined on the bash script and that is /root/Desktop/Finalyear/dec1.py

This python script performs the decryption of the re-encrypted files and after that store those decrypted files in the specified directory.

The python script behind this:

```
import os
import base64
import openpyxl
from openpyxl import load_workbook
from cryptography.fernet import Fernet

def decrypt_file(filename, key):
    output_filename = os.path.splitext(filename)[0]

    with open(filename, 'rb') as file:
        data = file.read()

    cipher_suite = Fernet(key)
    decrypted_data = cipher_suite.decrypt(data)

    with open(output_filename, 'wb') as output_file:
        output_file.write(decrypted_data)

    return output_filename
# Load encryption key from Excel file
excel_filename = input("Enter the path of the encryption key Excel file: ")
workbook = load_workbook(excel_filename, read_only=True)
sheet = workbook.active
key_str = sheet['B1'].value
print(key_str)
#key = base64.urlsafe_b64decode(key_str)
key = key_str
# Print the key for verification
print("Key:", key)
# Get input encrypted files
filepaths = []
num_files = int(input("Enter the number of files to decrypt: "))

for i in range(num_files):
    filepath = input(f"Enter the path of encrypted file {i+1}: ")
    filepaths.append(filepath)

# Decrypt files
decrypted_files = []
for filepath in filepaths:
    try:
        decrypted_file = decrypt_file(filepath, key)
        decrypted_files.append(decrypted_file)
```

```
print(f"Decrypted {filepath}")
    except ValueError:
        print(f"Failed to decrypt {filepath}. Invalid key.")

print("\nFiles decrypted:")
for decrypted_file in decrypted_files:
    print(decrypted_file)
```

Fig 6.7.3:-Python script to decrypt re-encrypted files

6.8 Merge The Decrypted Files

The seventh step is to Merge the decrypted files.

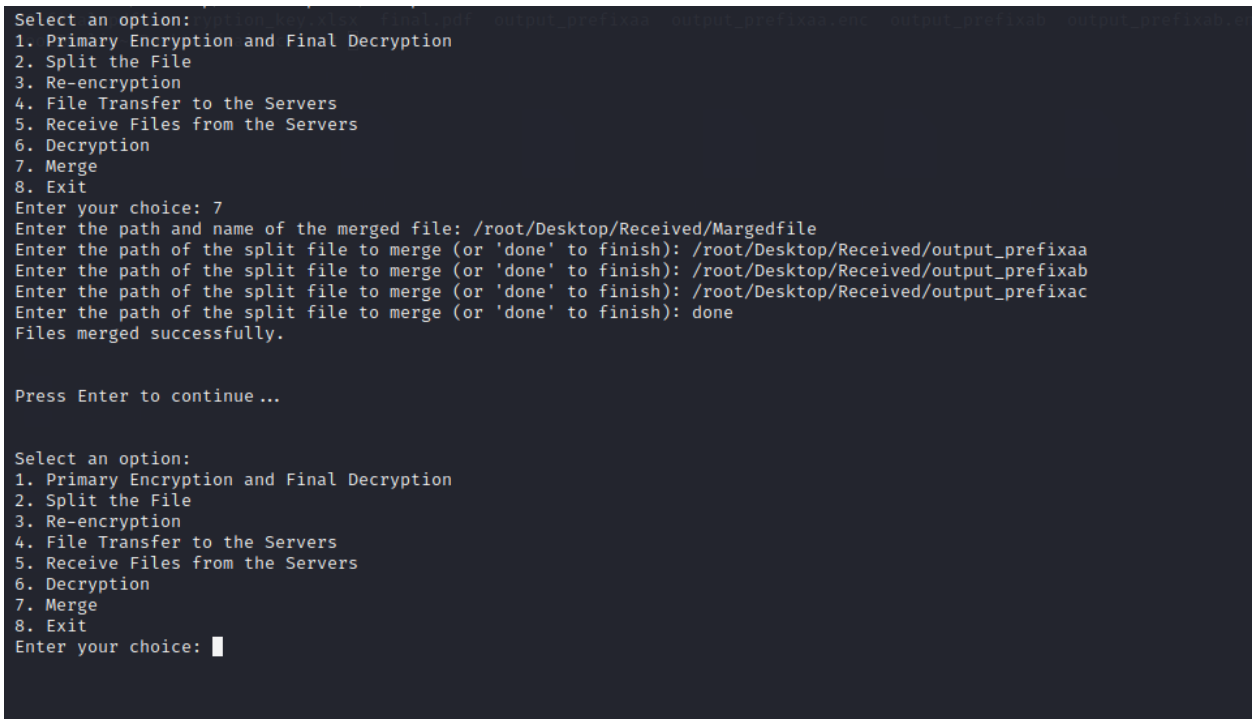


Fig 6.8.1:-Merge

To merge the decrypted files we have to enter option 7 on the bash script. Then we have to enter the path where we want to store the merged file and also we have to specify the name of the merged file. The path and name of the merged file is:

- /root/Desktop/Received/Mergedfile

Then we have to enter the path of the splitted files that we want to merge. The path:

- /roo/Desktop/Received/output_prefixaa
- /roo/Desktop/Received/output_prefixab
- /roo/Desktop/Received/output_prefixac

Then we have to click done to finish the loop.

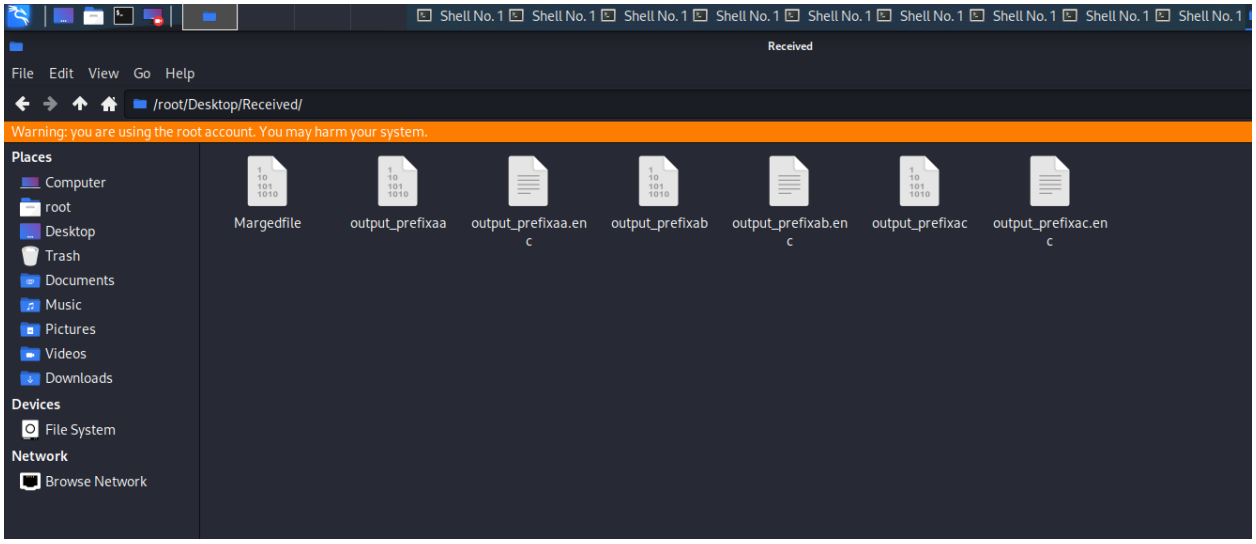


Fig 6.8.2:-Stored Merged files

Here from the above figure we can see that the Merged file is stored on the /root/Desktop/Received directory.

The merge script is:

```
# Function to execute merge script
execute_merge() {
    read -p "Enter the path and name of the merged file: " merged_file_path

    while true; do
        read -p "Enter the path of the split file to merge (or 'done' to finish): " split_file_path
        if [ "$split_file_path" = "done" ]; then
            break
        fi

        cat "$split_file_path"* >> "$merged_file_path"
    done

    echo "Files merged successfully."
}
```

Fig 6.8.2:-Megred script

Here from the above figure we can see that we have used a kali linux tool called cat here and integrate it with the bash script.

6.9 Final Decryption

This is the 8th step of this project to do the final Decryption.

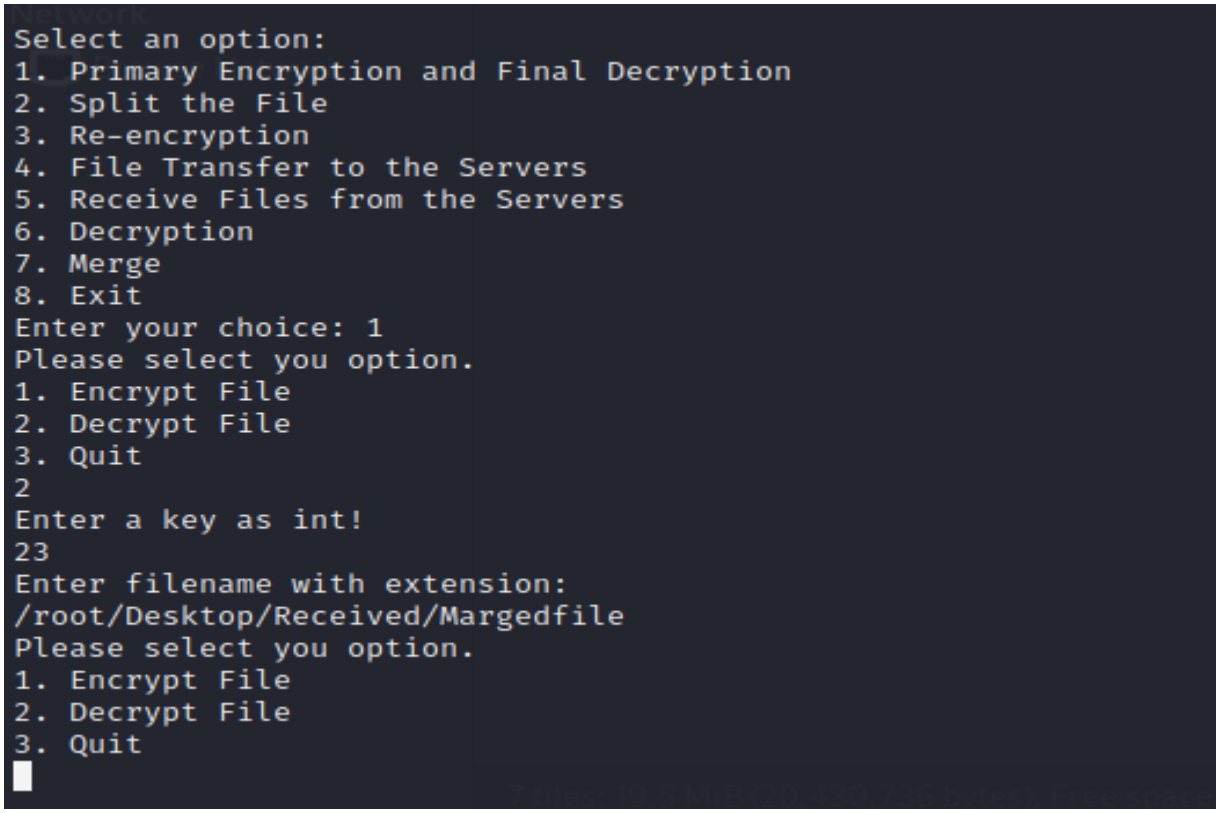


Fig 6.9.1: Final Decryption

To do this final decryption we have to select option 1 from the bash script then we have to select option 2 that is Decrypt File.

After that we have to enter the integer key that we used while encrypting the final.pdf file on primary level and that is 23.

Then we have to enter the path with the file name that we want to decrypt to get the original file that we used previously.

The path of the file is:

- /root/Desktop/Received/Margedfile

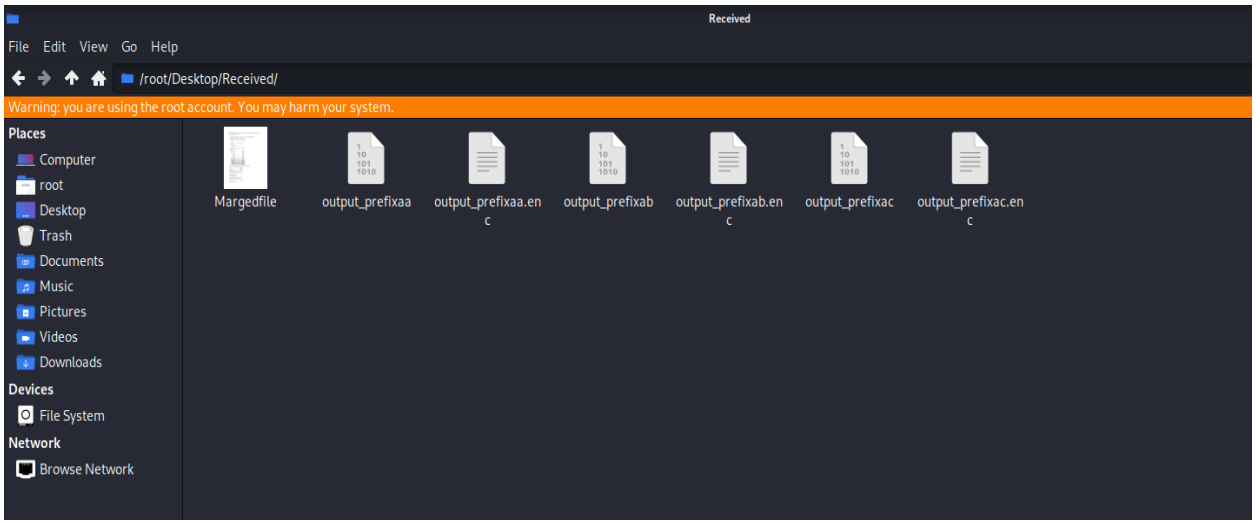


Fig 6.9.1:-Original File

After Merging the file we can see the Encrypted Margedfile has changed and we get the original pdf file that is Margedfile. If we enter this file we can see this is the exact same file that we used at first to encrypt that is final.pdf. So the final.pdf and the Margedfile are same we just rename this file while doing decryption.

The python script behind this name is hello.py . This python script is integrated with the bash script at execute_primary_encryption_decryption() function.

The path of the hello.py python script is defined on the bash script and that is /root/Desktop/Finalyear/hello.py function.



Conclusion and Future Studies

Topics

- **Conclusion**
- **Future Studies**

7.1 Conclusion

In conclusion a secure erasure code-based cloud storage system with secure data forwarding offers reliable data security and encrypted communication for cloud storage. Erasure coding and secure data transmission techniques are used to ensure the availability, durability, secrecy, and integrity of data. Improved fault tolerance, successful information recovery, and security against unwanted access and data breaches are all features of the system. This proposed system is much more reliable and secure from the typical third party cloud system. With the help of Erasure coding and Re-encryption scheme a user data is encrypted, splitted and re-encrypted before storing to the servers. It provides extra security of the data. Overall, it provides a safe and dependable way to store and retrieve data on the cloud.

7.2 Future Studies

In future we can concentrate on developing many areas of secure erasure code-based cloud storage systems with secure data forwarding systems with multiple functionalities. These include boosting system performance, providing data integrity and auditing capabilities, supporting multi-cloud environments, optimizing costs, enhancing user experience and accessibility, enabling advanced collaboration and sharing features and other things. We can add Data Lifecycle Management it includes data migration, disposal and other things. We can take extra security and privacy measures to make it more secure. These research topics attempt to solve the changing requirements and difficulties in cloud computing settings by improving the functionality, security, effectiveness, and usability of cloud storage systems.

References

[1] Z. Wilcox-O'Hearn and B. Warner, "Tahoe: The Least-Authority Filesystem," Proc. Fourth ACM Int'l Workshop Storage Security and Survivability (StorageSS), pp. 21-26, 2008.

[2] A. Adya, W.J. Bolosky, M. Castro, G. Cermak, R. Chaiken, J.R. Douceur, J. Howell, J.R. Lorch, M. Theimer, and R. Wattenhofer, "Farsite: Federated, Available, and Reliable Storage for an Incompletely Trusted Environment," Proc. Fifth Symp. Operating System Design and Implementation (OSDI), pp. 1-14, 2002.

[3] H.-Y. Lin and W.-G. Tzeng, "A Secure Decentralized Erasure Code for Distributed Network Storage," IEEE Trans. Parallel and Distributed Systems, vol. 21, no. 11, pp. 1586-1594, Nov. 2010.

[4] S.C. Rhea, P.R. Eaton, D. Geels, H. Weatherspoon, B.Y. Zhao, and J. Kubiatowicz, "Pond: The Oceanstore Prototype," Proc. Second USENIX Conf. File and Storage Technologies (FAST), pp. 1-14, 2003.

[5] M. Blaze, G. Bleumer, and M. Strauss, "Divertible Protocols and Atomic Proxy Cryptography," Proc. Int'l Conf. Theory and Application of Cryptographic Techniques (EUROCRYPT), pp. 127-144, 1998.

[6] J. Kubiatowicz, D. Bindel, Y. Chen, P. Eaton, D. Geels, R. Gummadi, S. Rhea, H. Weatherspoon, W. Weimer, C. Wells, and B. Zhao, "Oceanstore: An Architecture for Global-Scale Persistent Storage," Proc. Ninth Int'l Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 190- 201, 2000.

[7] M. Mambo and E. Okamoto, "Proxy Cryptosystems: Delegation of the Power to Decrypt Ciphertexts," IEICE Trans. Fundamentals of Electronics, Comm. and Computer Sciences, vol. E80-A, no. 1, pp. 54-63, 1997.

[8] P. Druschel and A. Rowstron, "PAST: A Large-Scale, Persistent Peer-to-Peer Storage Utility," Proc. Eighth Workshop Hot Topics in Operating System (HotOS VIII), pp. 75-80, 2001.

[9] A. Haeberlen, A. Mislove, and P. Druschel, "Glacier: Highly Durable, Decentralized Storage Despite Massive Correlated Failures," Proc. Second Symp. Networked Systems Design and Implementation (NSDI), pp. 143-158, 2005.

[10] A.G. Dimakis, V. Prabhakaran, and K. Ramchandran, “Ubiquitous Access to Distributed Data in Large-Scale Sensor Networks through Decentralized Erasure Codes,” Proc. Fourth Int’l Symp. Information Processing in Sensor Networks (IPSN), pp. 111-117, 2005.

[11] D.R. Brownbridge, L.F. Marshall, and B. Randell, “The Newcastle Connection or Unixes of the World Unite!,” Software Practice and Experience, vol. 12, no. 12, pp. 1147-1162, 1982

[12] R. Bhagwan, K. Tati, Y.-C. Cheng, S. Savage, and G.M. Voelker, “Total Recall: System Support for Automated Availability Management,” Proc. First Symp. Networked Systems Design and Implementation (NSDI), pp. 337- 350, 2004

[13] M. Kallahalla, E. Riedel, R. Swaminathan, Q. Wang, and K. Fu, “Plutus: Scalable Secure File Sharing on Untrusted Storage,” Proc. Second USENIX Conf. File and Storage Technologies (FAST), pp. 29- 42, 2003.

[14] A.G. Dimakis, V. Prabhakaran, and K. Ramchandran, “Decentralized Erasure Codes for Distributed Networked Storage,” IEEE Trans. Information Theory, vol. 52, no. 6 pp. 2809-2816, June 2006.